



Tideman

Problem set



شما از پیش یاد گرفتید که انتخابات جمعی نوع ساده‌ای از الگوریتم برای تشخیص برنده‌ی یک انتخابات می‌باشد: هر رای‌دهنده تنها می‌تواند به یک نامزد رای دهد و نامزدی که بیشترین رای را دریافت کند برنده می‌شود.

اما این روش از رای‌گیری ایراداتی هم دارد؛ برای مثال در نمونه زیر با سه نامزد چه اتفاقی می‌افتد؟

Ballot	Ballot	Ballot	Ballot	Ballot
Alice	Alice	Bob	Bob	Charlie

رای‌گیری جمعی در اینجا Alice و Bob را مساوی اعلام می‌کند. آیا این نتیجه درستی است؟

یک نوع دیگری از رای‌گیری به نام **ranked-choice system**، به رای‌دهندگان اجازه می‌دهد تا به بیشتر از یک نامزد رای دهند. در واقع به جای اینکه تنها به اولویت اول خود رای دهند، آنها می‌توانند به ترتیب ترجیح خود، به بیش از یک نفر رای دهند. نتیجه می‌تواند مثل نمونه زیر باشد:

Ballot	Ballot	Ballot	Ballot	Ballot
1. Alice 2. Bob 3. Charlie	1. Alice 2. Charlie 3. Bob	1. Bob 2. Alice 3. Charlie	1. Bob 2. Alice 3. Charlie	1. Charlie 2. Alice 3. Bob

اینجا هر رای‌دهنده نه تنها اولویت اول، بلکه اولویت سوم خودش را نیز مشخص کرده است. حالا انتخاباتی که قبلاً مساوی اعلام شده بود، میتواند یک برنده داشته باشد. در این انتخابات که در آن **Alice** و **Bob** مساوی بودند، حالا رای‌دهنده‌ای که به **Charlie** رای داده، **Alice** را به **Bob** ترجیح داده است، بنابراین **Alice** می‌تواند به عنوان برنده اعلام شود.

این روش می‌تواند یکی دیگر از نقاط ضعف رای‌گیری جمعی را نیز حل کند:

Ballot	Ballot	Ballot	Ballot	Ballot
1. Alice 2. Charlie 3. Bob	1. Alice 2. Charlie 3. Bob	1. Bob 2. Alice 3. Charlie	1. Bob 2. Alice 3. Charlie	1. Bob 2. Alice 3. Charlie
Ballot	Ballot	Ballot	Ballot	
1. Charlie 2. Alice 3. Bob	1. Charlie 2. Alice 3. Bob	1. Charlie 2. Alice 3. Bob	1. Charlie 2. Bob 3. Alice	

چه کسی باید برنده این انتخابات باشد؟ در روش رای‌گیری جمعی، **Charlie** با چهار رای نسبت به **Bob** با سه رای و **Alice** با دو رای، برنده می‌شود (اگر با رای‌گیری **instant runoff** هم آشنا باشید می‌دانید که **Charlie** در آن سیستم هم برنده است!).

Alice می‌تواند به طور معقولی به نتیجه اعتراض کند و ادعای برنده بودن کند: چون از 9 نفر، 5 نفر از رای‌دهندگان (اکثریت)، **Alice** را به **Charlie** ترجیح داده‌اند، پس بیشتر رای‌دهندگان از برد **Alice** راضی‌تر می‌شوند.

Alice در این انتخابات برنده‌ی به اصطلاح **Condorcet** می‌باشد: فردی که در مسابقه‌ی دو به دو با هرکدام از نامزدها برنده می‌شود. اگر این انتخابات فقط با حضور **Alice** و **Bob**، یا حتی **Alice** و **Charlie** بود، **Alice** برنده می‌شد.

سیستم رای‌گیری **Tideman** (یا زوج‌های رتبه‌بندی شده) یک نوع **ranked-choice voting** است که برد برنده **Condorcet** را تضمین می‌کند. در فایلی به اسم **tideman.c** در فولدری به اسم **tideman**، برنامه‌ای بنویسید که یک انتخابات با روش **Tideman** را شبیه‌سازی کند.

Distribution code را دانلود کنید.

در <https://cs50.dev> وارد شوید. روی ترمینال کلیک کرده و cd را به تنهایی اجرا کنید. ترمینال شما باید مثل نمونه زیر باشد:

```
$
```

حال دستور زیر را اجرا کنید تا فایل zip به اسم tideman.zip را دانلود کنید:

```
wget https://cdn.cs50.net/2024/fall/psets/3/tideman.zip
```

بعد با اجرا کردن دستور زیر فولدری به اسم tideman بسازید:

```
unzip tideman.zip
```

از اینجا به بعد دیگر نیازی به فایل zip ندارید پس با دستور زیر به همراه Y آن را حذف کنید:

```
rm tideman.zip
```

حالا دستور زیر را اجرا کرده تا به فولدر مورد نظر منتقل شوید:

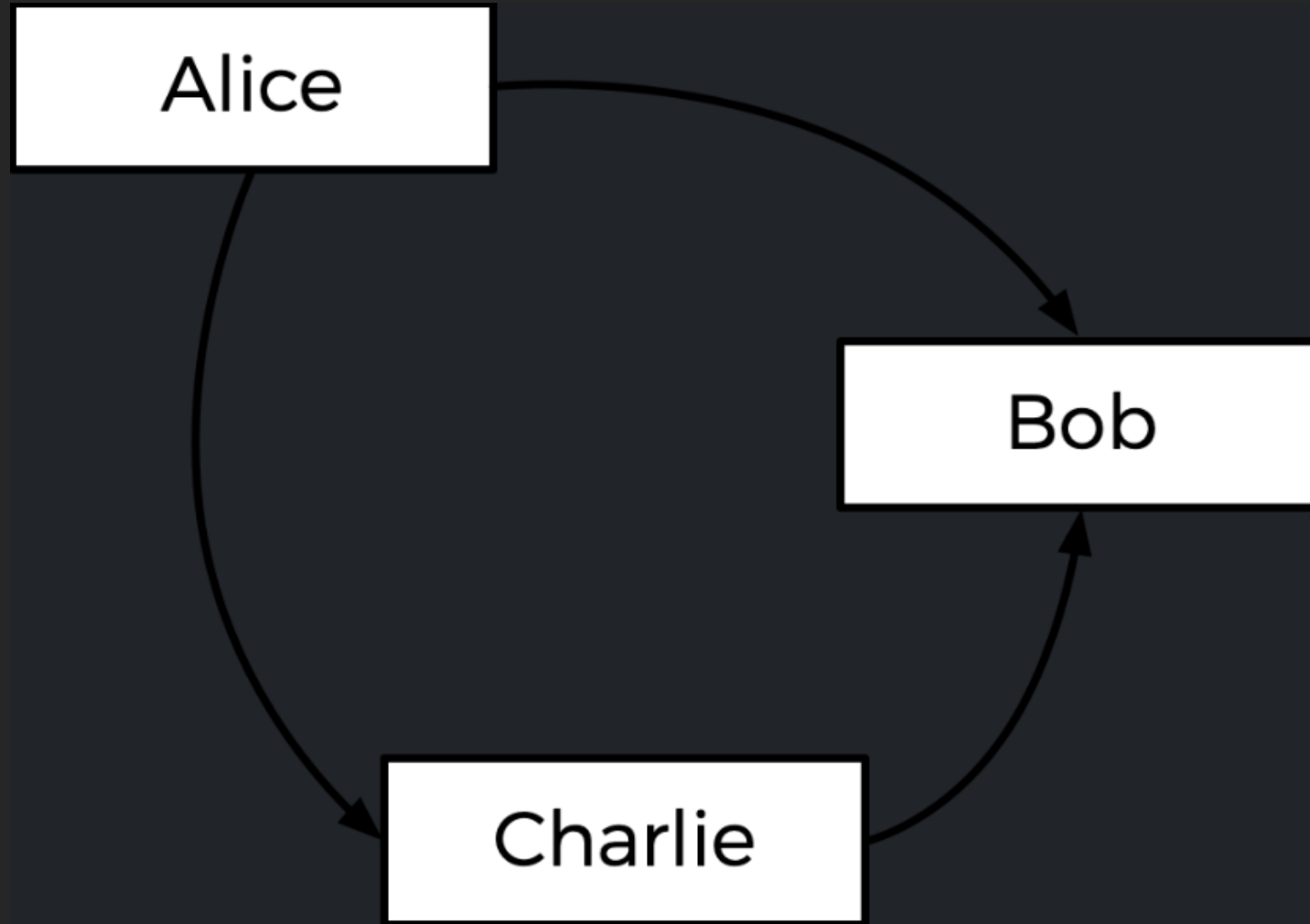
```
cd tideman
```

ترمینال شما باید شبیه نمونه زیر باشد:

```
tideman/ $
```

بعد از این مراحل با اجرا کردن `ls` میتوانید فایلی به اسم `tideman.c` را مشاهده کنید. با اجرا کردن دستور `code tideman.c` میتوانید این فایل را باز کرده و در آن شروع به نوشتن برنامه خود کنید. اگر چنین نشد به مراحل بالا برگردید و ببینید کجا را اشتباه کرده‌اید؟

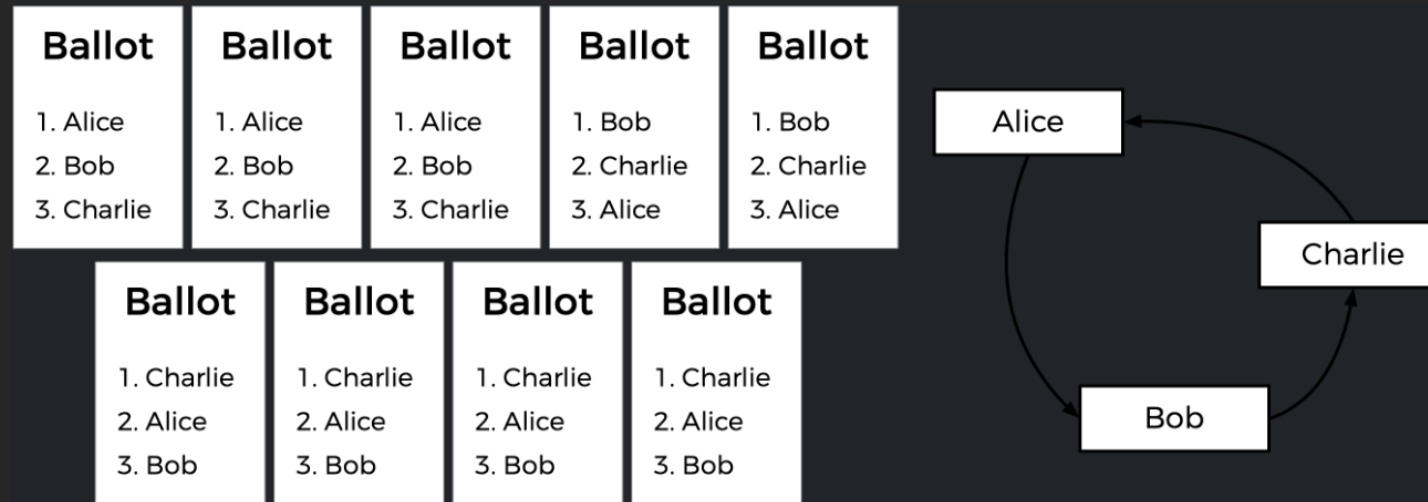
به طور کلی، روش Tideman با ساختن یک گراف از نامزدها کار می‌کند که در آن یک پیکان از نامزد A به نامزد B نشان می‌دهد که نامزد A در یک رقابت دو به دو، نامزد B را شکست داده است. گراف انتخابات بالا به شکل زیر خواهد بود:



پیکانی که از Alice به Bob وصل شده به معنی این است که رای‌دهندگان بیشتری Alice را به Bob ترجیح می‌دهند (5 نفر Alice و 4 نفر Bob را ترجیح می‌دهند). به همان شکل، پیکان‌های دیگر نشان می‌دهند که رای‌دهندگان بیشتری Alice را به Charlie و Charlie را به Bob ترجیح می‌دهند.

با نگاه کردن به این گراف، روش Tideman می‌گوید که مبدا این گراف باید برنده‌ی انتخابات باشد (نامزدی که هیچ پیکانی منتهی به او نباشد). در اینجا Alice مبدا این گراف است. (هیچ پیکانی سمت او نیست یعنی هیچکس در مسابقه دو به دو به او ترجیح داده نمی‌شود.) پس Alice در این انتخابات برنده اعلام می‌شود.

البته ممکن است که با کشیدن پیکان‌ها اصلاً برنده‌ی Condorcet وجود نداشته باشد. به نمونه‌ی زیر توجه کنید:



بین Alice و Bob، Alice با نسبت 7 رای به 2 رای به Bob ترجیح داده شده است. بین Bob و Charlie، Bob با نسبت 5 به 4 رای به Charlie ترجیح داده شده است؛ اما بین Charlie و Alice، Charlie با نسبت 6 به 3 رای به Alice ترجیح داده شده است. اگر گراف این انتخابات را بکشیم، هیچ مبدایی را مشاهده نمی‌کنیم! ما یک دور از نامزدها را داریم که Alice، Bob را می‌برد، و Charlie، Bob را می‌برد و Charlie، Alice را می‌برد (مثل بازی سنگ کاغذ قیچی). انگار که در اینجا نمی‌توانیم هیچ کس را برنده اعلام کنیم.

برای مدیریت این موضوع، الگوریتم Tideman باید مراقب باشد که در گراف نامزدها چرخه ایجاد نشود. چگونه این کار را انجام می‌دهد؟

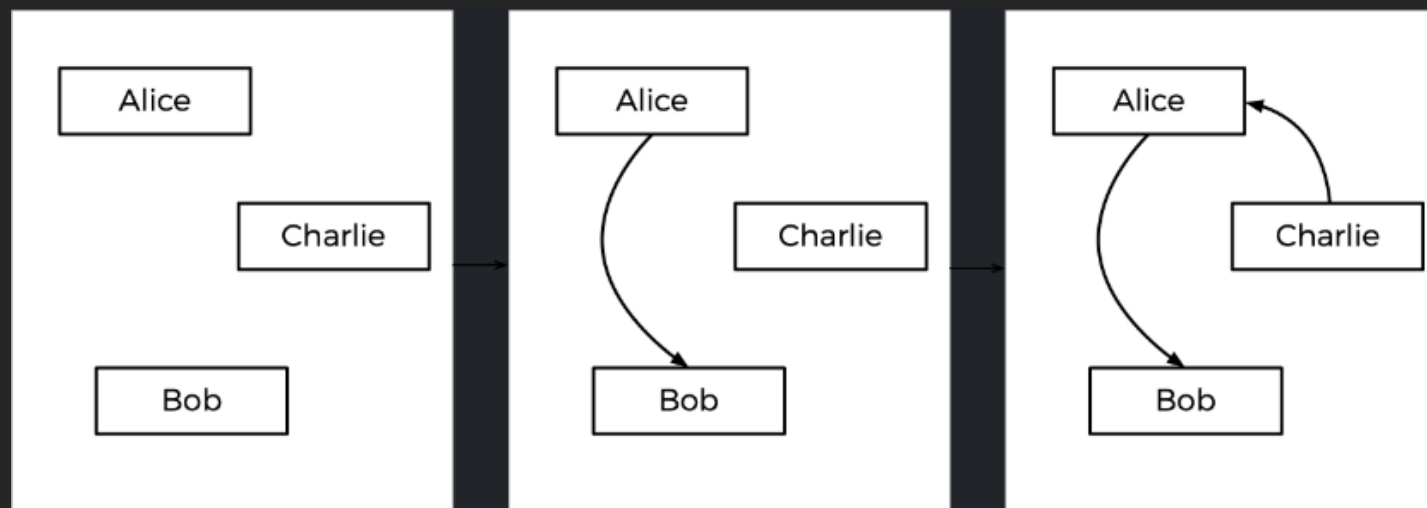
الگوریتم، ابتدا قوی‌ترین پیکان‌ها را انتخاب کرده و بر روی آن قفل می‌کند، زیرا این پیکان‌ها از نظر منطقی مهم‌ترین هستند. به طور مشخص، الگوریتم Tideman مشخص می‌کند که پیکان‌های رقابت باید به صورت جداگانه و بر اساس "قدرت" پیروزی به گراف اضافه شوند (هرچه تعداد افرادی که یک نامزد را به حریفش ترجیح می‌دهند بیشتر باشد، پیروزی قوی‌تر است).

تا زمانی که یک پیکان بدون ایجاد دور، قابل قفل شدن باشد، آن را به گراف اضافه می‌کنیم؛ در غیر این صورت، پیکان نادیده گرفته می‌شود.

این روش با رای‌های بالا چگونه کار می‌کند؟ بزرگ‌ترین نسبت برد برای Alice و Bob است: 7 نفر Alice را به Bob ترجیح داده‌اند (در هیچ رقابت دو به دوی دیگری برنده با بیشتر از 7 رای به کسی ترجیح داده نشده است). پس اول پیکان Alice-Bob در گراف قفل می‌شوند. دومین بزرگ‌ترین نسبت برد برای Alice و Charlie با نسبت 3-6 است، پس این پیکان هم قفل می‌شود.

بعدی برد Bob به Charlie با نسبت 4-5 است. توجه کنید که: اگر الان یک پیکان از Bob به Charlie وصل کنیم، یک دور تشکیل می‌دهیم! از آنجایی که در گراف نباید دور به وجود بیاوریم، این پیکان را رد می‌کنیم و اصلاً به گراف اضافه نمی‌کنیم. اگر پیکان‌های دیگر هم وجود داشتند، آن‌ها را هم بررسی می‌کنیم اما فعلاً این آخرین پیکان بود، پس گراف ما کامل شد.

مراحل بالا در زیر نشان داده شده است:



با توجه به نتایج گراف، Charlie منبع است (هیچ پیکانی به سمت او نیست). پس Charlie برنده اعلام می‌شود.

به طور رسمی تر، روش Tideman از سه بخش تشکیل شده:

- شمارش: پس از اینکه همه رأی دهندگان تمام ترجیحات خود را مشخص کردند، برای هر جفت نامزد تعیین کنید که کدام نامزد ترجیح داده شده است و با چه اختلافی این ترجیح وجود دارد.
 - مرتب سازی: جفت های نامزدها را بر اساس قدرت پیروزی، به ترتیب نزولی مرتب کنید، جایی که قدرت پیروزی به تعداد رأی دهندگانی که نامزد ارجح را انتخاب کرده اند تعریف می شود.
 - قفل کردن: با قوی ترین جفت شروع کرده و به ترتیب جفت نامزدها را بررسی کنید و هر جفت را در گراف نامزدها "قفل کنید"، به شرطی که قفل کردن آن جفت باعث ایجاد چرخه در گراف نشود.
- وقتی گراف کامل شود، منبع گراف برنده است. (کسی که هیچ پیکانی به سمتش نباشد.)

بیایید به `tideman.c` نگاهی بیندازیم.

اول به `preferences` که یک آرایه دو بعدی است دقت کنید. عدد صحیح `preferences[i][j]` تعداد رای‌دهندگانی که نامزد `i` را به نامزد `j` ترجیح داده‌اند، نشان می‌دهد.

در این فایل آرایه دو بعدی دیگری نیز به اسم `locked` تعریف شده است. این آرایه گراف نامزدها را نشان می‌دهد. `Locked` یک آرایه `Boolean` می‌باشد، پس اگر `locked[i][j]` ، `true` باشد به معنی پیکانی از نامزد `i` به نامزد `j` است. `False` هم به معنی نبود پیکان است (به این نوع از نشان دادن گراف، `adjacency matrix` می‌گویند).
مورد بعد یک `struct` به اسم `pair` است که برای یک جفت از نامزدها استفاده می‌شود: هر جفت شامل نمایه نامزد برنده (`winner`) و نامزد بازنده (`loser`) است.
نامزدها در آرایه `candidates` ذخیره می‌شوند که یک آرایه از رشته‌ها است. این آرایه اسم هر نامزد را نشان می‌دهد. آرایه `pairs` هم وجود دارد که جفت‌های نامزدها را نشان می‌دهد.

این برنامه، دو متغیر سراسری نیز دارد: `pair_count` و `candidate_count` که تعداد جفت‌ها و تعداد نامزدها در آرایه `pairs` و `candidates` را نشان می‌دهند.

حالا به تابع `main` نگاه کنید. دقت کنید که بعد از مشخص کردن تعداد نامزدها، برنامه در گراف `locked` دور می‌زند و همه مقادیر را به `false` تغییر می‌دهد که به معنی نبودن پیکان در این گراف می‌باشد.

سپس برنامه، حلقه‌ای برای جمع‌آوری ترجیحات رای‌دهندگان را طی می‌کند و آنها را در آرایه `ranks` (با فراخوانی `vote`)، ذخیره می‌کند. `ranks[i]` یک نمایه از نامزدی است که اولویت `i` ام یک رای‌دهنده است. این رتبه‌بندی‌ها به تابع `record_preference` داده می‌شوند که کارش گرفتن این رتبه‌بندی‌ها و به روزرسانی متغیر سراسری `preferences` است.

زمانی که همه `vote` ها جمع شدند، جفت‌های نامزدها به آرایه `pairs`، با فراخوانی `add_pairs` اضافه، با فراخوانی `sort_pairs` مرتب و با فراخوانی `lock_pairs` در گراف قفل می‌شوند. در آخر، `print_winner` فراخوانی می‌شود تا برنده‌ی انتخابات را چاپ کند.

در پایین فایل، توابع `vote`، `record_preference`، `add_pairs`، `sort_pairs`، `lock_pairs` و `print_winner` خالی‌اند و کامل کردن آنها بر عهده شماست!

c را طوری کامل کنید که یک انتخابات Tideman را شبیه‌سازی کند.

- تابع `vote` را کامل کنید.
- این تابع آرگومان‌های `rank`، `name` و `ranks` را دریافت می‌کند. اگر `name` با اسم یکی از نامزدهای معتبر یکی بود، باید `ranks` را به معنی اینکه یک رای‌دهنده این نامزد را به عنوان اولویت `rank` اش انتخاب کرده، بروز رسانی کنید. (0 اولویت اول و 1 اولویت دوم است)
- به یاد داشته باشید که `ranks[i]` نشان دهنده‌ی اولویت `i` ام کاربر است.
- اگر رتبه‌بندی به طور موفقیت‌آمیز ثبت شد، تابع باید `true` و در غیر این صورت `false` را برگرداند. (برای مثال `name` اسم هیچ یک از نامزدها نباشد)
- فرض کنید هیچ دو نامزدی اسم‌های یکسان نداشته باشند.
 - تابع `record_preferences` را کامل کنید.
- این تابع یک بار برای هر رأی‌دهنده فراخوانی می‌شود و آرایه `ranks` را به عنوان آرگومان دریافت می‌کند (به یاد داشته باشید که `ranks[i]` نشان‌دهنده ترجیح رتبه `i` ام رأی‌دهنده است، جایی که `ranks[0]` اولین ترجیح را نشان می‌دهد).

- تابع باید آرایه سراسری `preferences` را برای اضافه کردن ترجیحات رای‌دهنده فعلی به روزرسانی کند. به یاد داشته باشید که `preferences[i][j]` باید تعداد رای‌دهندگانی که نامزد `i` را به نامزد `j` ترجیح می‌دهند را نشان دهد.
- فرض کنید هر رای‌دهنده، تمام نامزدها را ردهبندی می‌کند.
 - تابع `add_pairs` را کامل کنید.
- این تابع باید تمام جفت نامزدهایی را که در آن یک نامزد بر دیگری ترجیح داده می‌شود، به آرایه `pairs` اضافه کند. جفت نامزدهایی که مساوی هستند (هیچ‌کدام بر دیگری ترجیح داده نمی‌شود) نباید به این آرایه اضافه شوند.
- این تابع باید متغیر سراسری `pair_count` را به تعداد جفت نامزدها به روز رسانی کند. (این جفت ها باید بین `pairs[0]` و `pairs[pair_count - 1]` ذخیره شوند)
 - تابع `sort_pairs` را کامل کنید.

- این تابع، باید آرایه `pairs` را به ترتیب نزولی قدرت پیروزی مرتب کند. قدرت پیروزی به شکل تعداد رای‌دهندگانی تعریف می‌شود که کاندیدای مرجح را ترجیح داده‌اند. اگر چند جفت کاندیدا قدرت پیروزی یکسانی داشته باشند، دیگر ترتیب آنها اهمیتی ندارد.
 - تابع `Lock_pairs` را کامل کنید.
 - این تابع باید گراف قفل‌شده را ایجاد کند و تمام پیکان‌ها را به ترتیب نزولی قدرت پیروزی اضافه کند، به شرطی که افزودن پیکان باعث ایجاد دور نشود.
 - تابع `print_winner` را کامل کنید.
 - این تابع باید اسم نامزدی که مبدا گراف است را چاپ کند. فرض کنید بیشتر از یک مبدا وجود نداشته باشد.
- شما نباید چیز دیگری را در `tideman.c` بجز `vote`، `record_preferences`، `add_pairs`، `sort_pairs`، `lock_pairs` و `print_winner` تغییر دهید. البته تا زمانی که توابع موجود دیگر در `tideman.c` تغییر نکنند، می‌توانید علاوه بر `header` های دیگر، توابع جدیدی نیز اضافه کنید.

برنامه خود را با نمونه های زیر تست کنید:

- یک انتخابات با تعداد دلخواه نامزد. (حداکثر 9 تا)
- رای دادن به یک نامزد با اسم او
- رای های نامعتبر برای نامزد هایی که در برگه رای وجود ندارند
- چاپ کردن برنده انتخابات

درستی

```
check50 cs50/problems/2025/x/tideman
```

استایل

```
style50 tideman.c
```

نحوه ارسال

```
submit50 cs50/problems/2025/x/tideman
```

CS50x Iran

Harvard's Computer Science 50x Iran

